Instituto Tecnológico de Costa Rica

Escuela de Ingeniería Electrónica

Taller de diseño digital (EL3313)

Prof. Kaleb Alfaro Bonilla, M.Sc.

Estudiantes:

2021155576 - Bolaños Solís Oscar

2022207101 - Bou Espinoza Raquel

2022195304 - Lobo Campos Andrés

2021021770 - Sánchez Herrera Angélica

Lab. 2: Microcontrolador

I Semestre 2026

Resumen

En el presente documento se desarrolla el diseño e implementación de un sistema basado en un núcleo RISC-V sobre una FPGA Nexys 4 DDR. El sistema integra una memoria de programa (ROM), una memoria de datos (RAM) y periféricos mapeados en memoria, incluyendo un módulo de comunicación UART, los cuales se interconectan mediante una arquitectura de memoria mapeada.

El procesador ejecuta instrucciones almacenadas en la ROM, permitiendo la interacción con los distintos dispositivos del sistema. En particular, se implementa la transmisión de datos hacia una computadora a través de la interfaz UART. Durante el desarrollo se abordaron aspectos clave como la decodificación de direcciones, la integración de periféricos y la sincronización de señales.

La correcta operación del sistema fue verificada mediante pruebas en hardware sobre la FPGA, complementadas con simulaciones en Vivado y el uso de la herramienta Tera Term para la observación de la comunicación serial.

Tabla de contenidos

Resumen 2
Introducción 4
Diseño del sistema 4
Descripción de módulos 5
Mapa de memoria 8
Funcionamiento del sistema 8
Implementación en FPGA 10
Verificación de funcionamiento en la FPGA 10
Conclusiones 10
Referencias 11

Introducción

En este laboratorio se desarrolló e implementó un sistema basado en un núcleo RISC-V de 32 bits sobre una FPGA Nexys 4 DDR. El sistema es capaz de ejecutar instrucciones almacenadas en memoria y comunicarse con una computadora mediante una interfaz UART.

El objetivo principal consiste en integrar los distintos módulos de hardware, incluyendo memoria, periféricos y lógica de control, para lograr la transmisión de datos desde la FPGA hacia una computadora, verificando su correcto funcionamiento mediante herramientas externas como TeraTerm.

Diseño del sistema

El sistema está compuesto por un núcleo de procesamiento PicoRV32, una memoria de programa (ROM), una memoria de datos (RAM) y periféricos mapeados en memoria, incluyendo un botón, un conjunto de LEDs y un módulo de comunicación UART.

La interacción entre estos módulos se realiza mediante una interfaz de memoria tipo bus, donde el procesador genera señales de control como mem_valid, mem_addr, mem_wdata, mem_wstrb y mem_ready, permitiendo el acceso a los distintos dispositivos a través de un esquema de memoria mapeada.

El módulo superior (top_microprocesador) se encarga de la decodificación de direcciones y de la gestión de las señales de control, seleccionando el dispositivo correspondiente (ROM, RAM o periféricos) según la dirección solicitada por el procesador.

El periférico UART permite la comunicación serial con una computadora, y está compuesto internamente por módulos de transmisión y recepción (uart_tx y uart_rx), así como buffers FIFO que permiten desacoplar la velocidad de la CPU y la interfaz serial, garantizando una transmisión de datos confiable.

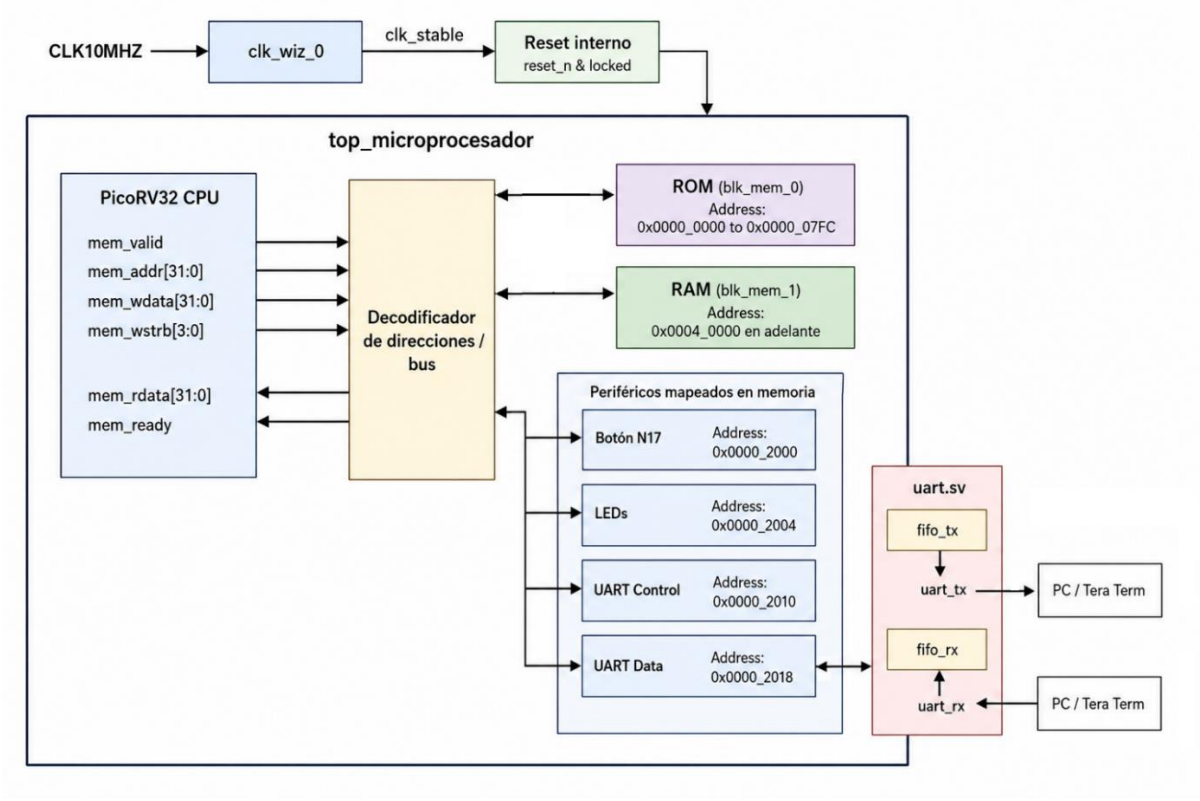

Figura1. Esquemático del microcontrolador

Descripción de módulos

En el siguiente enlace se puede acceder directamente al código completo de cada módulo en el repositorio de GitHub: https://github.com/Oscar-Electronics/Taller_diseño_digital/tree/main/Laboratorio%202

PicoRV32

El módulo PicoRV32 corresponde a un núcleo de procesamiento basado en la arquitectura RISC-V, diseñado para ser ligero y eficiente en implementaciones sobre FPGA. Este núcleo es el encargado de ejecutar el programa almacenado en la memoria de solo lectura (ROM) y de coordinar el funcionamiento general del sistema.

La comunicación del procesador con los demás módulos se realiza mediante una interfaz de memoria, utilizando señales como mem_valid, mem_ready, mem_addr, mem_wdata, mem_wstrb y mem_rdata. A través de estas señales, el procesador puede realizar operaciones de lectura y escritura sobre la memoria y los periféricos mapeados en memoria.

El procesador genera la dirección del acceso (mem_addr), los datos de escritura (mem_wdata) y las señales de control (mem_valid y mem_wstrb), mientras que los módulos externos responden con los datos leídos (mem_rdata) y la señal de confirmación (mem_ready). Este esquema permite una integración sencilla con memorias y periféricos dentro de un mismo espacio de direcciones.

De esta forma, el PicoRV32 actúa como el elemento central del sistema, controlando la ejecución del programa y gestionando la interacción con la memoria y los dispositivos externos.

ROM

La memoria de solo lectura (ROM) fue implementada utilizando el IP Block Memory Generator de Vivado. Esta memoria se encarga de almacenar el programa que será ejecutado por el núcleo PicoRV32.

La ROM es inicializada mediante un archivo de coeficientes (.coe), el cual contiene las instrucciones del programa en formato binario, previamente compiladas a partir del código fuente. Estas instrucciones son cargadas en la memoria durante la síntesis del diseño.

Dentro del sistema, la ROM se encuentra mapeada en el espacio de direcciones, específicamente en el rango 0x0000_0000 a 0x0000_07FC. El procesador accede a esta memoria a través de la interfaz de memoria, utilizando señales como mem_valid, mem_addr y mem_rdata para realizar operaciones de lectura.

Dado que se trata de una memoria de solo lectura, no se realizan operaciones de escritura sobre ella, por lo que únicamente responde a solicitudes de lectura del procesador, proporcionando las instrucciones necesarias para la ejecución del programa.

RAM

La memoria de acceso aleatorio (RAM) permite el almacenamiento temporal de datos durante la ejecución del programa, siendo utilizada por el procesador para variables, resultados intermedios y manejo de información dinámica.

Al igual que la ROM, la RAM fue implementada utilizando el IP Block Memory Generator de Vivado. Sin embargo, a diferencia de la ROM, esta memoria soporta tanto operaciones de lectura como de escritura.

Dentro del sistema, la RAM se encuentra mapeada en el espacio de direcciones a partir de la dirección 0x0004_0000. El acceso a esta memoria se realiza mediante la interfaz de memoria del

procesador, utilizando señales como mem_valid, mem_addr, mem_wdata, mem_wstrb, mem_rdata y mem_ready.

Cuando el procesador realiza una operación de escritura, envía los datos a través de mem_wdata junto con las señales de control correspondientes, mientras que en las operaciones de lectura la RAM responde proporcionando los datos solicitados mediante mem_rdata. Este mecanismo permite un intercambio eficiente de información entre el procesador y la memoria durante la ejecución del programa.

UART

El módulo UART permite la comunicación serial entre la FPGA y una computadora, facilitando el envío y recepción de datos a través de una interfaz asíncrona. En este sistema, la UART se configura con una tasa de transmisión de 9600 baudios.

El periférico es accedido mediante un esquema de memoria mapeada, utilizando dos direcciones principales: una para control y estado (0x0000_2010) y otra para el intercambio de datos (0x0000_2018). A través de estas direcciones, el procesador puede enviar y recibir información de manera sencilla mediante operaciones de lectura y escritura.

Internamente, el módulo UART está compuesto por un transmisor (uart_tx), un receptor (uart_rx) y buffers FIFO para transmisión y recepción. El transmisor se encarga de convertir datos paralelos en una secuencia serial, mientras que el receptor realiza la operación inversa. Los buffers FIFO permiten desacoplar la velocidad del procesador de la velocidad de transmisión serial, evitando la pérdida de datos.

De esta forma, la UART actúa como una interfaz de comunicación eficiente entre el sistema embebido y el entorno externo, permitiendo la interacción con herramientas como terminales seriales en la computadora.

Módulo Top microprocesador

El módulo superior (top_microprocesador) es el encargado de integrar todos los componentes del sistema, incluyendo el núcleo de procesamiento PicoRV32, la memoria ROM, la memoria RAM y los periféricos mapeados en memoria, como el botón, los LEDs y la UART.

Este módulo actúa como intermediario entre el procesador y los diferentes dispositivos, implementando la decodificación de direcciones a partir de la señal mem_addr. Con base en esta dirección, el módulo selecciona el dispositivo correspondiente (ROM, RAM o periféricos) y habilita las señales de control necesarias para realizar operaciones de lectura o escritura.

Asimismo, el módulo gestiona la interfaz de memoria del sistema, propagando las señales mem_valid, mem_wdata y mem_wstrb hacia los módulos seleccionados, y retornando las señales mem_rdata y mem_ready al procesador, garantizando una comunicación correcta y sincronizada.

Adicionalmente, el módulo superior integra la lógica de reloj y reset del sistema, utilizando un bloque de generación de reloj (Clock Wizard) para obtener una señal estable, así como lógica de reset que asegura una correcta inicialización de todos los componentes.

De esta manera, el top_microprocesador constituye el núcleo de interconexión del sistema, coordinando la ejecución del procesador y la interacción con la memoria y los periféricos.

Mapa de memoria

El sistema utiliza un esquema de memoria mapeada, donde cada periférico ocupa un rango específico de direcciones dentro del espacio de memoria del procesador.

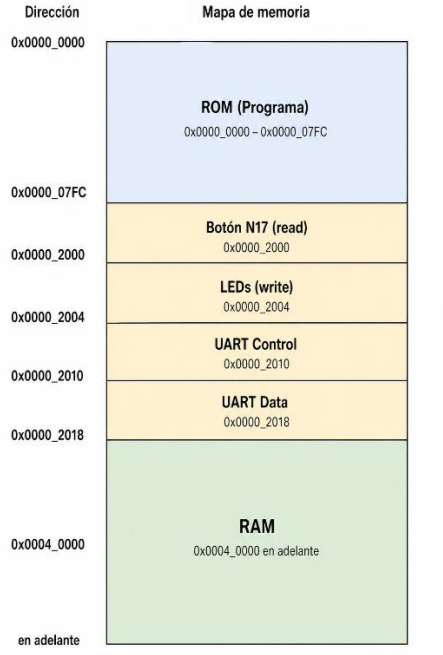

Figura 2. Mapa de memoria

Funcionamiento del sistema

Al iniciar el sistema, el procesador PicoRV32 comienza la ejecución del programa almacenado en la memoria ROM, accediendo a las instrucciones mediante la interfaz de memoria. Estas

instrucciones definen el comportamiento del sistema, incluyendo la interacción con los periféricos mapeados en memoria.

La comunicación entre el procesador y los distintos módulos se realiza mediante la interfaz de memoria, utilizando señales como mem_valid, mem_addr, mem_wdata, mem_wstrb, mem_rdata y mem_ready. Cuando el procesador desea acceder a un dispositivo, activa la señal mem_valid junto con la dirección correspondiente, y espera la confirmación de la operación mediante mem_ready.

El módulo superior se encarga de decodificar la dirección generada por el procesador y seleccionar el dispositivo adecuado, ya sea la ROM, la RAM o algún periférico. En el caso de operaciones de lectura, el dispositivo responde con los datos a través de mem_rdata, mientras que en las operaciones de escritura el procesador envía los datos mediante mem_wdata.

En particular, para la comunicación serial, el procesador escribe datos en el registro de datos de la UART, lo que provoca que el módulo de transmisión (uart_tx) envíe la información hacia la computadora. Asimismo, los datos recibidos pueden ser almacenados en el sistema a través del módulo de recepción (uart_rx).

De esta manera, el sistema permite la ejecución de un programa almacenado en ROM y la interacción con dispositivos externos mediante un esquema de memoria mapeada, garantizando una comunicación eficiente entre el procesador, la memoria y los periféricos.

Implementación en FPGA

El diseño fue implementado en una FPGA Nexys 4 DDR utilizando la herramienta Vivado. Se integraron módulos en Verilog y SystemVerilog para el resto del sistema.
Las restricciones de pines fueron definidas mediante el archivo de constraints .xdc, incluyendo la configuración del reloj, la interfaz UART y señales de control.
Las siguientes imágenes muestran el correcto funcionamiento de la simulación del programa en Vivado. Se prueba mediante un testbench donde se puede ver que las señales se comportan como deberían, mem_valid oscila entre 1 y 0, mem_addr muestra los valores y la uart recibe un 1 que indica que en el TeraTerm se debería ver una A al presionar el botón de reset cpu.

Verificación de funcionamiento en la FPGA

Una vez obtenido el resultado esperado en el osciloscopio de Vivado, se procede a generar el bitstream y la implementación en la FPGA. En el siguiente enlace se puede verificar la funcionalidad de la implementación realizada:
Enlace del video: https://youtube.com/shorts/lD_0mbMSNYE?feature=share

Conclusiones

Se logró implementar exitosamente un sistema basado en un núcleo RISC-V capaz de ejecutar instrucciones y comunicarse mediante UART. Uno de los principales retos fue la implementación del protocolo AXI-Lite y la conexión con el .coe. La aplicación de la calculadora en el .coe fue el

reto principal. Este proyecto permitió comprender la integración de hardware digital complejo, así como la interacción entre procesadores y periféricos en arquitecturas embebidas.

Referencias

[1] Digilent, «digilent-xdc/Nexys-4-DDR-Master.xdc at master · Digilent/digilent-xdc», *GitHub*. https://github.com/Digilent/digilent-xdc/blob/master/Nexys-4-DDR-Master.xdc

[2] Oskarwires, «GitHub - oskarwires/uart2: SystemVerilog UART transciever», GitHub. https://github.com/oskarwires/uart2.git